📋 Enunciado: Exercício Prático de Persistência de Dados e SQLAlchemy (CRUD de Produtos)

🎯 Objetivo

Praticar o desenvolvimento de aplicações Python conectadas a bancos de dados relacionais utilizando a biblioteca **SQLAlchemy** (ORM). O aluno deverá implementar um sistema completo de gerenciamento de produtos no terminal, contemplando todas as operações do ciclo **CRUD** (*Create, Read, Update, Delete*), além de validações de entrada e busca por diferentes chaves (ID e Descrição).

🏗️ Descrição do Cenário

Você foi contratado por uma empresa de comércio varejista para desenvolver um protótipo de sistema de controle de estoque via linha de comando (CLI). O sistema precisa persistir as informações dos produtos de forma segura em um banco de dados SQLite local utilizando mapeamento objeto-relacional (ORM).

O sistema deve disponibilizar um menu interativo contendo as seguintes funcionalidades obrigatórias:

1. **Adicionar Produto:**
   - Deve solicitar ao usuário a **descrição**, a **quantidade** (validando obrigatoriamente para números inteiros) e o **valor** do produto (validando para números decimais/float).
   - Deve salvar o registro de forma persistente no banco de dados.
2. **Listar Produtos:**
   - Deve consultar todos os registros cadastrados na tabela e exibi-los formatados na tela, exibindo ID, descrição, quantidade e valor formatado em reais (R$).
3. **Atualizar Produto (por Descrição ou por ID):**
   - O sistema deve permitir a atualização dos dados de um produto buscando-o tanto pela sua **descrição** quanto pelo seu **ID**.
   - Caso o produto seja encontrado, o sistema deve permitir alterar a descrição, quantidade ou valor. Se o usuário pressionar Enter sem digitar nada ou inserir dados inválidos, os valores anteriores devem ser mantidos de forma resiliente.
4. **Deletar Produto (por Descrição ou por ID):**
   - O sistema deve permitir a exclusão de um produto do banco de dados utilizando como critério de busca a **descrição** ou o **ID**.
   - Deve emitir mensagens de sucesso ou avisar caso o registro não seja encontrado.
5. **Estrutura Técnica e Tecnologias Obrigatórias:**
   - Utilizar a biblioteca SQLAlchemy para criação do engine, gerenciamento de sessões

(sessionmaker) e mapeamento declarativo (declarative_base).

- A tabela de banco de dados deve se chamar produto e conter as seguintes colunas:
  - id: Chave primária inteira com autoincremento (autoincrement=True).
  - descricao: String de até 100 caracteres, não nula.
  - quantidade: Inteiro, não nulo.
  - valor: Ponto flutuante (Float), não nulo.

↗️ **Instruções para Envio da Atividade:** Cada estudante deverá submeter pelo Blackboard (sistema AVA da instituição) exclusivamente o código-fonte desenvolvido para a solução (arquivo .py). O arquivo individual deve ser renomeado obrigatoriamente seguindo o formato padrão produto_nomedoaluno.py (substituindo pelo seu nome completo) para fins de identificação e correção automatizada ou manual.

✅Critérios de Avaliação e Desempenho

- **Conceito A (Atendimento Pleno + Destaque):** Atribuição dada aos estudantes que entregaram a lógica de programação impecável, o diagrama de classes completo e avançaram na arquitetura de arquivos separados ou no uso avançado de bibliotecas nativas.

- **Conceito B (Atendimento Pleno):** Projetos com códigos totalmente funcionais e brilhantes, que cumpriram a lógica complexa, mas cujas notas foram ajustadas devido à ausência do Diagrama de Classes obrigatório na etapa inicial.

- **Conceito C (Atendimento Parcial):** Atividades que apresentaram códigos logicamente corretos com testes funcionando, mas que sofreram penalizações pela falta da documentação visual exigida e por pequenos desvios de padronização nas nomenclaturas de métodos.

- **Fatores de Atenção e Penalização:** O principal foco de perda de pontos foi o esquecimento da modelagem visual, o desvio do escopo proposto e a ausência de blindagem e encapsulamento adequados nos atributos dos objetos.